LOW-LEVEL DESIGN (LLD)

Smart Track App

Architecture • Modules • APIs • Data Design • Security

Organized technical specification

Table of Contents

1. Introduction
2. System Modules & Components
3. Database Scheme Details
4. Key Algorithm & Logic
5. Interface & Screen Flow
6. References

1. Introduction

1.1 Scope of the Document

This Low- Level Design (LLD) document provides detailed component specifications, module logic, database schema design, algorithm definitions, and user interface workflows for Smart Track — a web-based retail management and transaction-tracking application for micro-retailers, such as mini-marts and sari-sari stores. It serves as a development guide for implementation, coding, and unit testing.

1.2 Intended Audience

- Developers and programmers — implement modules, database tables, and business logic.implement modules, database tables, and business logic.
- System testers — prepare test cases based on described functions and flows.
- Project instructors and reviewers — verify design completeness and implementation details.

1.3 System Overview

Smart Track consists of five core functional modules that work together:

- User Management: Handles account registration, login authentication, and access control.
- Inventory Management: Manages product records, pricing, stock levels, and product information.
- Sales & Transactions: Records sales, automatically updates stock, and generates transaction history.
- Customer Credit Management: Tracks customer debts and repayment with balance updates.
- Dashboard & Reports: Summarizes daily operations including stock status, sales totals, and outstanding balances.

2. System Modules & Components

2.1 User Management

Purpose: Handle system users, login, and access control

Key Functions:

- Store Assistant - stores staff data: ID, full name, username, password
- Login - verify credentials and start user session
- Logout - end current user session

Process Flow - Login:

1. Users enters username and password
2. System checks database for matching record
3. Match found - create session - redirect to Dashboard
4. No match - display error message

2.2 Product / Inventory Management

Purpose: Track products, stock levels, pricing, and reorder points

Key Functions:

- Product — stores product data: ID, name, category, price, stock quantity, reorder level
- Add Product — insert new product record
- Update Product — edit product details and stock
- Check Low Stock — flag items where stock falls below reorder level

Process Flow - Add Product:

1. User fills in product details
2. System validates inputs
3. Save record to database
4. Show success message and refresh product list

2.3 Sales / Transaction Management

Purpose: Record sales transactions, update stock, and handle debt payments

Key Functions:

- Sale — stores transaction data: sale ID, staff, customer, total, payment method, date
- Sale Item — line items: products sold, quantities, prices at time of sale
- Create Sale — record transaction, reduce stock, and create debt if on credit
- Record Payment — log partial or full payments toward outstanding debt

Process Flow - Make a sale:

1. Select products and enter quantities
2. System checks if stock is sufficient - Not enough stock → show "Insufficient Stock" - Enough stock → proceed
3. Calculate subtotals and total amount
4. Record sale details and reduce product stock
5. If payment method = Debt → create new debt record
6. Return sale confirmation

2.4 Customer Credit / Debt Management

Purpose: Track customer debts and record payments

Key Functions:

- Debt — stores credit record: debt ID, customer name, contact number, total amount, balance, status
- Debt Payment — stores payment record: payment ID, debt ID, amount paid, date
- Record Debt — create new debt record when payment is deferred
- Record Payment — update debt balance and status when customer pays
- Get Outstanding Debts — list all unpaid or partially paid debts

Process Flow - Record Debt Payment:

1. Select existing debt record
2. Enter payment amount
3. Add amount to total paid and subtract from remaining balance
4. Save payment record with current date
5. Update debt status:
    - Balance = 0 → Status = "Paid"
    - Balance > 0 but amount paid > 0 → Status = "Partial"
    - No payment yet → Status = "Unpaid"

2.5 Dashboard & Reports

Purpose: Display summary information and generate business reports

Key Functions:

- Get Low Stock Items — list products where stock falls below reorder level
- Get Total Sales — calculate total sales within a selected date range
- Get Total Outstanding Debts — sum of all customer balances not yet paid
- Get Daily Sales Summary — show sales totals per day for the last 7 days

Process Flow - View Low Stock Alert:

1. System compares each product's stock quantity against its reorder level
2. Collect all products where stock ≤ reorder_level
3. Display results on dashboard alert section

3. Database Scheme Details

Database Schema

1. Store Assistants (`store_assistants`)

assistant_id — INT; PRIMARY KEY, AUTO_INCREMENT. Unique staff ID.

full_name — VARCHAR(100); NOT NULL. Staff full name.

username — VARCHAR(50); NOT NULL, UNIQUE. Login username.

password — VARCHAR(255); NOT NULL. Hashed password.

2. Products (`products`)

product_id — INT; PRIMARY KEY, AUTO_INCREMENT. Unique product ID.

product_name — VARCHAR(100); NOT NULL. Product name.

category — VARCHAR(50). Product category.

price — DECIMAL(10,2); NOT NULL. Selling price per unit.

stock_quantity — INT; NOT NULL, DEFAULT 0. Current stock count.

reorder_level — INT; NOT NULL, DEFAULT 5. Low stock alert level.

created_at — TIMESTAMP; DEFAULT CURRENT_TIMESTAMP. Date the product was added to the system.

3. Sales (`sales`)

sale_id — INT; PRIMARY KEY, AUTO_INCREMENT. Unique transaction ID.

assistant_id — INT; FOREIGN KEY REFERENCES `store_assistants(assistant_id)`. Staff member who processed the sale.

customer_name — VARCHAR(100). Name of the customer.

total_amount — DECIMAL(10,2); NOT NULL. Total sale amount.

amount_paid — DECIMAL(10,2); NOT NULL. Cash received from the customer.

change_amount — DECIMAL(10,2); NOT NULL. Change given to the customer.

payment_method — ENUM('Cash', 'Debt'); NOT NULL. How the sale was paid.

sale_date — TIMESTAMP; DEFAULT CURRENT_TIMESTAMP. Date and time of the sale.

4. Sale Items (`sale_items`)

sale_item_id — INT; PRIMARY KEY, AUTO_INCREMENT. Unique line item ID.

sale_id — INT; FOREIGN KEY REFERENCES `sales(sale_id)`. Related transaction ID.

product_id — INT; FOREIGN KEY REFERENCES `products(product_id)`. Product sold.

quantity — INT; NOT NULL. Number of units sold.

unit_price — DECIMAL(10,2); NOT NULL. Price per unit at the time of sale.

subtotal — DECIMAL(10,2); NOT NULL. Quantity multiplied by unit price.

5. Debts (`debts`)

debt_id — INT; PRIMARY KEY, AUTO_INCREMENT. Unique debt record ID.

customer_name — VARCHAR(100); NOT NULL. Customer name.

contact_number — VARCHAR(20). Customer phone number.

sale_id — INT; FOREIGN KEY REFERENCES `sales(sale_id)`. Related sale transaction.

total_debt — DECIMAL(10,2); NOT NULL. Full amount owed.

amount_paid — DECIMAL(10,2); DEFAULT 0.00. Total payments made so far.

balance — DECIMAL(10,2); NOT NULL. Remaining amount to pay.

status — ENUM('Unpaid', 'Partial', 'Paid'); NOT NULL, DEFAULT 'Unpaid'. Current payment status.

debt_date — TIMESTAMP; DEFAULT CURRENT_TIMESTAMP. Date the debt was created.

6. Debt Payments (`debt_payments`)

payment_id — INT; PRIMARY KEY, AUTO_INCREMENT. Unique payment record ID.

debt_id — INT; FOREIGN KEY REFERENCES `debts(debt_id)`. Related debt being paid.

amount_paid — DECIMAL(10,2); NOT NULL. Amount paid in this payment.

payment_date — TIMESTAMP; DEFAULT CURRENT_TIMESTAMP. Date the payment was recorded.

4. Key Algorithm & Logic

This section describes the algorithms and business logic used in Smart Track to process store transactions, calculate cashflow, manage inventory, track customer credit, and generate business reports. These algorithms ensure that financial records and inventory information remain accurate and consistent.

4.1 Store Management Algorithm

Purpose: To manage multiple store locations under a single user account and ensure that the system displays information for the currently selected store.

Process Flow:

1. The user logs in using valid credentials.
1. The system retrieves the store locations associated with the user's account.
1. The system displays the available stores in the Store Switcher.
1. The user selects a store location.
1. The system sets the selected store as the active store.
1. The system retrieves the transactions, inventory records, cash accounts, and customer credit records associated with that store.
1. The system updates the active store name in the navigation bar and breadcrumb trail.

Expected Output: The dashboard and its modules display information belonging to the selected store.

4.2 Cashflow Calculation Algorithm

Purpose: To calculate daily net cashflow by comparing the total recorded income and expenses.
Formula:
Net Cashflow = Total Income − Total Expenses
Process Flow:

1. The system retrieves the income and expense transactions for the active store for the current day.
1. The system separates transactions into income and expense categories.
1. The system calculates the total income.
1. The system calculates the total expenses.
1. The system subtracts total expenses from total income.
1. The system displays the resulting net cashflow in the Dashboard summary card.
1. When a new transaction is recorded, the system recalculates the values and refreshes the dashboard.

Example:

- Total Income: ₱12,480.00
- Total Expenses: ₱3,840.00
- Net Cashflow: ₱8,640.00

Expected Output: The Dashboard displays the current day's net cashflow based on recorded transactions.

4.3 Cash on Hand Calculation Algorithm

Purpose: To calculate the total available funds across physical cash and e-wallet accounts.
Formula:
Cash on Hand = Physical Cash Balance + E-Wallet Balance
Process Flow:

1. The system retrieves the cash account balances for the active store.
1.

The system identifies the physical cash balance.

1.
1.

The system retrieves the balances of the store's e-wallet accounts.

1.
1.

The system adds the physical cash balance and applicable e-wallet balances.

1.
1.

The system displays the total in the Cash on Hand summary card.

1.
1.

When a transaction affects an account balance, the system updates the relevant balance and recalculates the total.

1.

Example:

-

Physical Cash: ₱18,500.00

-
-

E-Wallet Balance: ₱5,820.00

-
-

Total Cash on Hand: ₱24,320.00

-

Expected Output: The Dashboard displays the combined balance of the store's physical cash and e-wallet accounts.

4.4 Inventory Asset Value Calculation Algorithm

Purpose: To calculate the total cost value of the store's inventory and determine the number of active products.

Formula:

Inventory Asset Value = Sum of (Stock Quantity × Cost Price) for All Active Products

Process Flow:

1.

The system retrieves the active products for the selected store.

1.
1.

The system obtains each product's stock quantity and cost price.

1.
1.

The system multiplies the stock quantity by the cost price for each product.

1.
1.

The system adds the values of all active products.

1.
1.

The system counts the active products or SKUs.

1.
1.

The system displays the total inventory asset value and active product count on the Dashboard.

1.

Example:

-

Total Inventory Asset Value: ₱86,450.00

-
-

Active Products: 248

-

Expected Output: The Dashboard displays the total inventory value at cost price and the number of active products.

4.5 Sales and Transaction Processing Algorithm

Purpose: To record sales, calculate item subtotals, determine customer change, and update inventory records.

Process Flow:

1.

The store assistant opens the Sales or Transactions interface.

1.
1.

The assistant selects the products and enters the quantities purchased.

1.
1.

The system checks whether sufficient stock is available for each product.

1.
1.

If the stock is insufficient, the system displays an error message and prevents the sale from proceeding.

1.
1.

If stock is sufficient, the system calculates the subtotal for each item.

1.
1.

The system calculates the total transaction amount.

1.
1.

The assistant enters the amount received from the customer and selects the payment method.

1.
1.

For a cash payment, the system calculates the customer's change.

1.

Formula:

Item Subtotal = Quantity × Selling Price
Total Amount = Sum of All Item Subtotals
Customer Change = Amount Received − Total Amount

1.

The system validates that the payment amount is sufficient for the sale.

1.
1.

The system records the transaction details, including the date, time, transaction type, category, payment method, and description.

1.
1.

The system reduces the stock quantities of the products sold.

1.
1.

The system updates the relevant financial records.

1.
1.

The system displays a transaction confirmation.

1.

Expected Output: The sale is recorded, customer change is calculated when applicable, and inventory quantities are updated.

4.6 Customer Credit (Utang) Management Algorithm

Purpose: To record customer debts, track repayment histories, and maintain accurate outstanding balances.

Process Flow:

1.

The store assistant opens the Store Credit (Utang) module.

1.
1.

The assistant selects an existing customer or records a new customer debt.

1.
1.

The system records the customer name, contact information when available, original debt amount, and current balance.

1.
1.

When a customer makes a payment, the assistant selects the debt record and enters the payment amount.

1.
1.

The system validates that the payment amount is greater than zero and does not exceed the outstanding balance.

1.
1.

The system calculates the remaining balance.

1.

Formula:

Remaining Balance = Previous Balance − Payment Amount

1.

The system records the payment and its date in the payment history.

1.
1.

The system updates the debt status:

1.
    - **Unpaid:** No payment has been made.
    - **Partially Paid:** A payment has been made, but a balance remains.
    - **Paid:** The remaining balance is zero.
1.

The system displays the updated debt balance and status.

1.

Expected Output: Customer debt records, payment histories, and remaining balances are maintained accurately.

4.7 Cashflow Analytics and Reporting Algorithm

Purpose: To generate cashflow summaries and display income and expenses over a selected period.
Process Flow:

1.

The user opens the Dashboard or Reports module.

1.
1.

The user selects a reporting period, such as the last seven days.

1.
1.

The system retrieves the relevant transactions for the active store and selected dates.

1.
1.

The system groups income and expense transactions by date.

1.
1.

The system calculates the total income and total expenses for the selected period.

1.
1.

The system prepares daily income and expense values for the dual-bar chart.

1.
1.

The system displays the chart and aggregate totals.

1.
1.

The system refreshes the results when the user changes the selected reporting period.

1.

Expected Output: The user can compare daily income and expenses and review aggregate totals for the selected period.

4.8 Dashboard Transaction Summary Algorithm

Purpose: To display recent transactions and the total number of transactions recorded for the current day.

Process Flow:

1.
The system retrieves transactions belonging to the active store.
1.
1.
The system filters the records to identify transactions recorded on the current day.
1.
1.
The system counts the day's transactions.
1.
1.
The system retrieves the most recent transactions for display.
1.
1.
The system organizes the transaction details into a table containing date and time, type, category, payment method, and description.
1.
1.
The system displays the transaction count and recent transaction records on the Dashboard.
1.
1.
When a new transaction is recorded, the system refreshes the count and transaction list.
1.

Expected Output: The Dashboard displays the latest transaction records and the current day's transaction count.

4.9 Data Integrity and Transaction Handling

Purpose: To maintain consistency among transaction records, inventory quantities, cash account balances, and customer credit records.

Process Flow:

1.
The system receives a request to record a sale, expense, or customer credit payment.
1.
1.
The system validates the submitted information.
1.
1.
The system begins a database transaction for operations requiring multiple related updates.
1.
1.

The system saves the transaction and updates the corresponding inventory, cash account, or credit records.

1.
1.

The system checks whether all required operations have completed successfully.

1.
1.

If all operations succeed, the system commits the database transaction.

1.
1.

If an operation fails, the system rolls back the related changes.

1.
1.

The system returns a success or error response to the user interface.

1.

Expected Output: Related financial and inventory records remain consistent, preventing incomplete updates when a multi-step operation fails.

5. Interface & Screen Flow

This section describes the main user interfaces of Smart Track, their components, and the navigation flow between modules. The application uses a persistent left navigation sidebar and a main workspace for displaying dashboard information and managing store operations.

5.1 Login Screen

Purpose: To authenticate store assistants and authorized users before granting access to Smart Track.
Main Components:

-

Username field

-
-

Password field

-
-

Login button

-
-

Error message display

-

Screen Flow:

1.

The user opens the Smart Track website.

1.
1.

The system displays the Login screen.

1.
1.

The user enters a username and password.

1.
1.

The system validates the credentials.

1.
1.

If the credentials are valid, the system retrieves the user's permitted store locations and opens the Dashboard.

1.
1.

If the credentials are invalid, the system displays an error message.

1.

5.2 Main Dashboard Screen

Purpose: To provide an overview of the selected store's cashflow, inventory value, and recent transactions.

Main Components:

-

Persistent left navigation sidebar

-
-

Store Switcher

-
-

Active store name and breadcrumb trail

-
-

Today's Net Cashflow summary card

-
-

Cash on Hand summary card

-
-

Total Inventory Asset Value summary card

-
-

Daily Cash Flow dual-bar chart

-
-

Date range filter

-
-

Recent Transactions table

-
-

Daily transaction counter

-

Screen Flow:

1.

The user logs in successfully.

1.
1.

The system loads the Dashboard for the active store.

1.
1.

The system retrieves the store's latest financial, inventory, and transaction data.

1.
1.

The summary cards display net cashflow, total cash on hand, and inventory asset value.

1.
1.

The daily cashflow chart displays income and expenses for the selected period.

1.
1.

The Recent Transactions table displays the latest recorded transactions.

1.
1.

The user selects another module from the sidebar to perform additional operations.

1.

5.3 Store Switcher Interface

Purpose: To allow users with access to multiple stores to select the store whose information they want to manage.

Main Components:

-

Store selection dropdown or selector

-
-

Store names and locations

-
-

Active store indicator

-

Screen Flow:

1.

The user selects the Store Switcher.

1.
1.

The system displays the available store locations.

1.
1.

The user selects a store.

1.
1.

The system updates the active store.

1.
1.

The Dashboard and other modules load records associated with the selected store.

1.
1.

The active store name appears in the navigation bar and breadcrumb trail.

1.

5.4 Cashflow (In/Out) Screen

Purpose: To record and review store income and expenses.
Main Components:

-

Income and expense transaction form

-
-

Transaction amount field

-
-

Category selection

-
-

Payment method selection

-
-

Description field

-
-

Transaction date and time

-
-

Transaction history table

-
-

Cashflow totals

-

Screen Flow:

1.

The user selects Cashflow (In/Out) from the sidebar.

1.
1.

The system displays the cashflow interface.

1.
1.

The user chooses whether to record income or an expense.

1.
1.

The user enters the amount, category, payment method, and description.

1.
1.

The system validates the transaction details.

1.

1.

The system saves the transaction.

1.

1.

The system updates the applicable financial summaries.

1.

1.

The transaction appears in the transaction history.

1.

5.5 Inventory & Assets Screen

Purpose: To manage product information, stock quantities, prices, and inventory asset values.

Main Components:

-

Product list

-

-

Product search

-

-

Product name and category

-

-

Cost price

-

-

Selling price

-

-

Stock quantity

-

-

Reorder level

-

-

Add Product and Update Product controls

-

-

Low-stock indicators

-

-

Inventory asset value summary

-

Screen Flow:

1.

The user selects Inventory & Assets from the sidebar.

1.

1.

The system displays the products associated with the active store.

1.
1.

The user adds a product or selects an existing product.

1.
1.

The user enters or updates the product details.

1.
1.

The system validates the submitted information.

1.
1.

The system saves the changes to the database.

1.
1.

The system refreshes the product list and recalculates the inventory asset value.

1.
1.

The system identifies products whose stock quantities are at or below their reorder levels.

1.

5.6 Store Credit (Utang) Screen

Purpose: To manage customer debts, monitor balances, and record repayments.
Main Components:

-

Customer name

-
-

Customer contact information, when available

-
-

Original debt amount

-
-

Remaining balance

-
-

Debt status

-
-

Payment history

-
-

Record Payment control

-
-

Outstanding debt list

-

Screen Flow:

1.

The user selects Store Credit (Utang) from the sidebar.

1.
1.

The system displays customer credit records.

1.
1.

The user selects a customer record or creates a new debt record.

1.
1.

The system displays the original amount, outstanding balance, and payment history.

1.
1.

When a payment is received, the user enters the payment amount.

1.
1.

The system validates and records the payment.

1.
1.

The system recalculates the balance and updates the debt status.

1.
1.

The updated information appears in the customer credit interface.

1.

5.7 Reports Screen

Purpose: To provide detailed information about sales performance, income, expenses, and outstanding customer debts.

Main Components:

-

Date range selector

-
-

Income and expense summaries

-
-

Daily cashflow chart

-
-

Transaction records

-
-

Sales performance summaries

-
-

Outstanding customer credit information

-

Screen Flow:

1.

The user selects Reports from the sidebar.

1.
1.

The system displays the available reporting options.

1.
1.

The user selects the reporting period.

1.
1.

The system retrieves the relevant records for the active store.

1.
1.

The system calculates the totals and groups transactions by date.

1.
1.

The system displays the results through charts, tables, and summary information.

1.

5.8 Settings Screen

Purpose: To provide access to account, store branch, and system configuration options.
Main Components:

-

Account information

-
-

Store information

-
-

Branch configuration options

-
-

System settings

-
-

Logout option

-

Screen Flow:

1.

The user selects Settings from the sidebar.

1.
1.

The system displays the available configuration options.

1.
1.

The user views or updates the settings permitted by their access level.

1.
1.

The system validates and saves any changes.

1.
1.

The user may return to the Dashboard or log out.

1.

5.9 Overall System Navigation Flow

The general navigation flow of Smart Track is:
Login → Store Selection → Dashboard → Selected Module → Perform Operation → Validate Input → Save Record → Update Display
The main modules accessible through the persistent sidebar are:

-

Dashboard

-
-

Cashflow (In/Out)

-
-

Inventory & Assets

-
-

Store Credit (Utang)

-
-

Reports

-
-

Settings

-

The Store Switcher allows users with access to multiple locations to change the active store. All relevant dashboard summaries and module records must correspond to the currently selected store. The interface uses Philippine Peso (₱) as the default currency representation. Income and positive cashflow use green or teal indicators, while expenses and cash outflows use orange or red indicators, following the supplied SRS.

6. References

1.
1.
1.

RESTful API Design Standards and HTTP/1.1 Specifications, RFC 7231.

1.
1.

Database ACID Compliance and Relational Database Design Principles.

1.